

IN4050 - Assignment 1

Børge Bjørnstadjordet - (Borgebj)

Content

How to run.....	1
Implementation	2
Exhaustive Search.....	3
Hill Climbing.....	4
Genetic Algorithm	5

How to run

All files use argument parser (`utility.py/parse_args`), requiring command line arguments.

For help: ````<file>.py -help````

To run a file:

````<file>.py -cities <#>````

Where:

<file> is one of the main files below, and

<#> is a number between 2 to 24 (how many cities)

Main files include (run) + example

Exhaustive_search.py	<code>```exhaustive_search.py -cities 10```</code>	
Hill_climb.py	<code>```hill_climb.py -cities 24```</code>	
Genetic.py	<code>```genetic.py -cities 10```</code> <code>```genetic.py -cities 24 -seed 61943```</code>	(2 options)

Support files (don't run)

- Plotter.py
  - o Functions that plot given data using pyplot
- Statistics.py
  - o Calculate statistics for additional information
- Utility.py
  - o Functions used by all, mainly for data management, e.g. finding distance, argument parser

## Implementation

Representation of cities throughout all algorithms was done by using indices, meaning first city = index 0 and so on.

For **exhaustive search**, I passed a range of numbers (1, 2, 3, 4, ...24) and generated all permutations. Then iterated over all and kept the best one.

For hill climb, I started by generating a random solution (using shuffle). Then through a while loop, i:

1. Generated neighbors by swapping every pair of cities once
2. Finds lowest distance among new neighborhoods
3. If its lowest, continue. If not, terminate and end.

For **genetic algorithms** I chose a strategy that bases itself on a generational model, essentially replacing most individuals in the population each generation up to a certain specified generation has been reached.

- For survivor selection I did **elitism** mixed with offspring creation through OX:
  - o 10% of best individuals weren't kept each generation.
- For **parent selection**, I chose tournament selection. For each parent, picking k random among population, returning best.
- For crossover operator, I chose order selection (OX), as it was easy to implement and effective.
- At the end of each generation, the best individual was saved to be looked at for next generation.
- Termination criteria were reaching '*max\_generations*' based on N (cities)

For calculating statistics, I did this in a separate file (statistics.py), which does things such as:

- Avg calculation, finding best, finding worst, standard deviation calculations, measuring runtime, collecting fitness values for plotting ...

## Exhaustive Search

The shortest tour among the first 10 cities is the tour:

**Copenhagen > Hamburg > Brussels > Dublin > Barcelona > Istanbul > Bucharest > Budapest > Berlin > Copenhagen**

With the distance of **7486.3100**

Time taken for 10 cities: **3.54 seconds**

## Statistics

(*plot\_extrapolation* in *plotter.py* and *extrapolate\_exhaustive* in *utils.py*)

Estimating time could be done by calculating the time used for each permutation together with how many cities, then averaging all values for an approximate constant factor.

Cities (n)	Time (y) in seconds	n!	$k_I = 1/n!$
5	0.0001	120	$\approx 8.33e-7$
6	0.0003	720	$\approx 4.17e-7$
7	0.0023	5040	$\approx 4.56e-7$
8	0.0188	40320	$\approx 4.66e-7$
9	0.1690	362880	$\approx 4.65e-7$
10	1.6896	3628800	$\approx 4.65e-7$

Constant factor is the average of the sum of all  $k_i$ ,  $\rightarrow \text{avg}(\text{sum}(k_i))$

$K \approx 5.1e-7$  seconds per permutation

$$\text{Time}(n) = 5.17e - 7 * n!$$

For  $n = 24$

$$\text{Time}(24) = (5.17e - 7) * 24! \approx 3.2e17 \text{ seconds total}$$

3.2e17 seconds is approximately 1.02e10 years

- It will take roughly **10 billion years** to permute 24 cities for TSP

(*statistics.py/plot\_extrapolation*)

```
=== Extrapolation Model ===
Time(x) = 4.656172277014932e-07 * x!
```

# Hill Climbing

The shortest tour among the first 10 cities is the tour:

**Copenhagen > Berlin > Budapest > Bucharest > Istanbul > Belgrade > Barcelona > Dublin > Brussels > Hamburg > Copenhagen**

With the distance of **7486.3100**

Time taken for **10** cities: **0.0000s** distance **7486.31** (best confirmed by exhaustive)

Time taken for **15** cities: **0.0020s** distance **10434.89**

Time taken for **20** cities: **0.0041s** distance **12774.78**

Time taken for **24** cities: **0.0013s** distance **12776.87**

Hill climbing performs significantly better than exhaustive search. Time taken for increasing cities barely change at all, while distance accuracy most often is good.

## Statistics

([statistics.py/run\\_statistics](#))

20 runs for 10 cities, and 24 cities.

```

=== Statistics for Hill Climb 10 ===
Worst distance over 20 runs: 8377.2400
Mean distance over 20 runs: 7613.4630
Best distance over 20 runs: 7486.3100
Standard deviation over 20 runs: 272.1056

=== Statistics for Hill Climb 24 ===
Worst distance over 20 runs: 16117.4900
Mean distance over 20 runs: 14382.3590
Best distance over 20 runs: 12719.0800
Standard deviation over 20 runs: 835.7696

```



# Genetic Algorithm

## Statistics

(**statistics.py/run\_statistics**)

20 runs for 24 cities with 3 different populations (40, 120, 360)

```

=== Statistics for Genetic 24/40 ===
Worst distance over 20 runs: 14232.3800
Mean distance over 20 runs: 13240.6440
Best distance over 20 runs: 12554.8600
Standard deviation over 20 runs: 474.1425

=== Statistics for Genetic 24/120 ===
Worst distance over 20 runs: 13697.7000
Mean distance over 20 runs: 12642.5360
Best distance over 20 runs: 12287.0700
Standard deviation over 20 runs: 432.8233

=== Statistics for Genetic 24/360 ===
Worst distance over 20 runs: 12864.8600
Mean distance over 20 runs: 12540.7640
Best distance over 20 runs: 12287.0700
Standard deviation over 20 runs: 205.0488

```

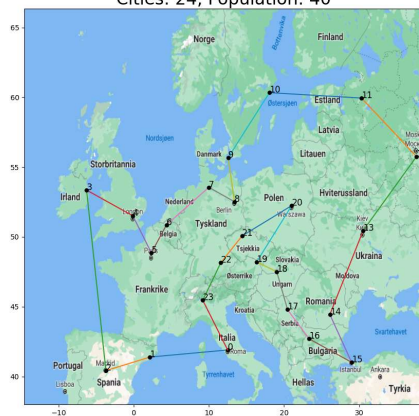
Best = 12287.0700

Worst = 14232

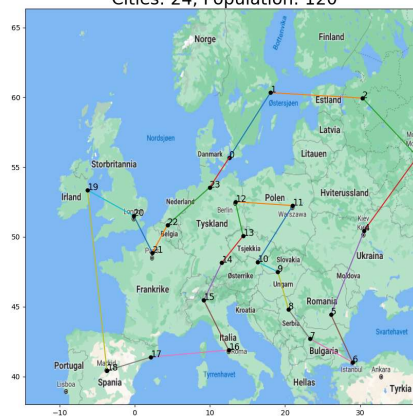
Best SD: 205.0488

Worst SD= 474.1425

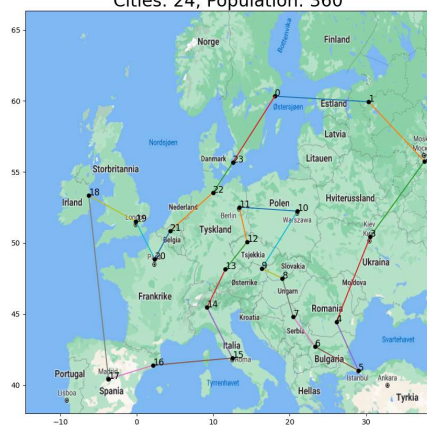
Cities: 24, Population: 40



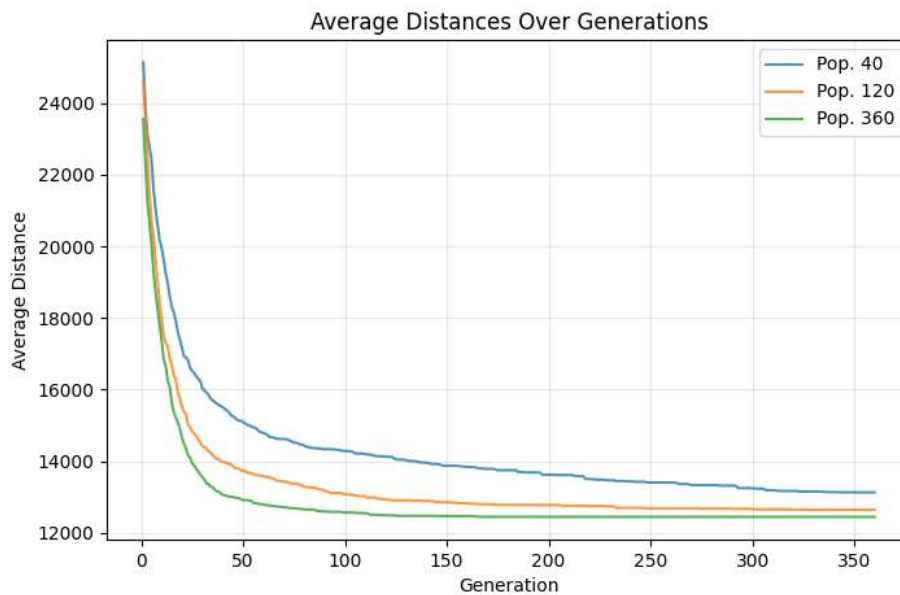
Cities: 24, Population: 120



Cities: 24, Population: 360



## Fitness over generations



- The fitness plot all shows relatively good results, whereas the bigger the population, the faster the algorithm converges with an optimal solution.
- In this case, the best parameters included 24 cities and a population of 360.
- The tradeoff with increasing population is the increasing time it takes to finish running.

NB!

- In code, fitness is defined as  $\left(\frac{1}{\text{DISTANCE}}\right)$ , this is to ensure lower distance = higher fitness
- In the above plot, I have included distance instead as it shows more clearly.

## GA questions

### [1] How did genetic results compare to exhaustive results for 10 cities?

Exhaustive search for 10 cities found best distance: **7486.3100**

Genetic search for 10 cities found best distance: **7486.3100**

The genetic found the best distance for 10 cities, but with small variations between runs, meaning not always the best.

### [2] Runtime comparison for 10 and 24 cities between genetic and exhaustive

Exhaustive

- 10 cities: 3.511 seconds
- 24 cities: ~10 billion years

Genetic

- 10 cities: 0.042 seconds
- 24 cities: 0.509 seconds

Conclusion:

- The speed of genetics is significantly faster than exhaustive, with results coming quite close to best result.

### [3] How many tours inspected by GA compared to exhaustive search?

- Exhaustive search creates every permutation, which is a huge number for 24.
  - $10! = 3\,628\,800$
  - $24! = 620\,448\,401\,733\,239\,439\,360\,000$
- Genetic algorithm inspects tours based on the population size specified, which means not every possible solution is checked.
  - 10 cities
    - Population size =  $\max(50, 10 \cdot 5) = 50$
    - Max generations =  $\max(50, 10 \cdot 15) = 150$
    - Overall tours inspected =  $50 \cdot 150 = 7500$  tours
  - 24 cities
    - Population size =  $\max(50, 24 \cdot 5) = 120$
    - Max generations =  $\max(50, 24 \cdot 15) = 360$
    - Overall tours inspected =  $120 \cdot 360 = 43\,200$  tours
  - These numbers are only “upper bound”, but in reality, not every solution from each generation is inspected, this is due to:
    - **Tournament selection**, selecting based on randomness and best fit
    - **Crossover**, producing offspring that may be duplicated solutions
    - **Mutations**, sometimes producing repeated paths

